



Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

COS132 - Imperative Programming

Copyright ©2026 by the 2026 staff of COS 132. All rights reserved.

Practical 4: Functions and Conditions

Release Date: 27-03-2026 at 23:59

Due Date: 12-04-2026 at 23:59

There is no late deadline for this practical.

Total Marks: 100

**Read the entire specification before starting
with the practical.**

Contents

1	General Instructions	4
2	Overview	5
2.1	Mark Breakdown	5
3	Background	5
4	Your Task:	5
4.1	HealthMetrics	6
4.1.1	Functions	6
4.1.1.1	double bmi(double weightKg, double heightM)	6
4.1.1.2	double bmr(int age, double weightKg, double heightM)	6
4.1.1.3	double tdee(double bmrValue, int activityLevel)	6
4.1.1.4	int bmiCategory(double bmiValue)	6
4.2	Constants	6
4.2.1	Variables	7
4.2.1.1	const string KJ	7
4.2.1.2	const string PROTEIN	7
4.2.1.3	const string CARBS	7
4.2.1.4	const string FAT	7
4.3	Meal	7
4.3.1	Functions	7
4.3.1.1	string makeMeal(const string name, double kilojoules, double proteinG, double carbsG, double fatG)	7
4.3.1.2	string getField(const string meal, const string field)	7
4.3.1.3	double getAmount(const string meal, const string field)	8
4.3.1.4	double kjPerGram(const string macro)	8
4.3.1.5	void printMealInfo(const string meal)	8
4.3.1.6	double macroRatio(const string meal, const string macro)	8
4.3.1.7	bool isHighMacro(const string meal, const string macro)	8
4.4	Workout	9
4.4.1	Functions	9
4.4.1.1	void printSetSummary(int reps, double weightKg)	9
4.4.1.2	bool isSetValid(int reps, double weightKg)	9
4.4.1.3	char repCategory(int reps)	9
4.4.1.4	int scoreFromCategory(char category)	9
4.4.1.5	int intensityLevel(int reps, double weightKg)	10
4.4.1.6	int progressionDecision(double prevWeight, double currWeight, int prevReps, int currReps)	10
4.4.1.7	int progressionDecision(int prevReps, int currReps)	10
4.5	Gym	11

4.5.1	Functions	11
4.5.1.1	string goalAdvice(int, double, double, string, string, string, int, char, double, double, int, int)	11
4.5.1.2	bool isGoalValid(char goal)	11
4.5.1.3	double kiloJouleTarget(double tdeeValue, char goal)	11
4.5.1.4	double proteinTarget(double weightKg, char goal)	11
4.5.1.5	int nutritionScore(int age, double weightKg, double heightM, const string m1, const string m2, const string m3, int activityLevel, char goal)	12
5	Implementation Details	12
6	Upload Checklist	13
7	Submission	13

1 General Instructions

- *Read the entire assignment thoroughly before you begin coding.*
- This assignment should be completed individually.
- Every submission will be inspected with the help of dedicated plagiarism detection software.
- Be ready to upload your assignment well before the deadline. There is a late deadline which is 48 hours after the initial deadline which has a penalty of 20% of your achieved mark. **No extensions will be granted.**
- If your code does not compile, you will be awarded a mark of 0. The output of your program will be primarily considered for marks, although internal structure may also be tested (eg. the presence/absence of certain functions or structure).
- Failure of your program to successfully exit will result in a mark of 0.
- Note that plagiarism is considered a very serious offence. Plagiarism will not be tolerated, and disciplinary action will be taken against offending students. Please refer to the University of Pretoria's plagiarism page at <https://portal.cs.up.ac.za/files/departamental-guide/>.
- Unless otherwise stated, the usage of non-C++98 features or additional libraries outside of those indicated in the assignment will **not** be allowed. Some of the appropriate files that you have submit will be overwritten during marking to ensure compliance to these requirements. **Please ensure you use C++98**
- All functions should be implemented in the corresponding `cpp` file. No inline implementation in the header file apart from the provided functions.
- The usage of ChatGPT and other AI-Related software to generate submitted code is strictly forbidden and will be considered as plagiarism.
- The use of this practical, in any form, by any company/business/organisation outside the University of Pretoria is strictly forbidden. This includes, but is not limited to, the use of the practical for discussion sessions, review sessions, marketing tools, or providing certain aspects as a free service.

2 Overview

In this practical, you will implement a basic gym simulator using functions and conditional (selection) statements. The program will calculate personalised protein targets, evaluate meal composition using macronutrients, and compute nutrition scores. It will also track workout progression and provide advice that integrates BMI, nutrition and workout intensity.

2.1 Mark Breakdown

The following table represents the total marks assigned to each task on FitchFork:

Task	Mark
HealthMetrics	5
Meal	16
Workout	24
Gym	26
Simulator	5
Conditions	24
Total	100

3 Background

Body Mass Index (BMI) is a widely used measure to classify body weight relative to height. It provides a quick assessment of whether an individual falls within underweight, normal, overweight, or obese ranges. Total Daily Energy Expenditure (TDEE) estimates the total number of kilojoules (calories) a person burns in a day, accounting for basal metabolic rate, daily activity and exercise. TDEE is critical in determining how much energy an individual needs to maintain, lose or gain weight. For the sake of simplicity we will only use BMR (Basal Metabolic Rate) and activity level to calculate it. Macronutrients (protein, carbohydrates and fats) or “macros” are essential components of a balanced diet. Protein, in particular, is vital for muscle repair and growth, making it a key factor for those engaged in resistance training or other high-intensity workouts.

4 Your Task:

You are required to implement the functions laid out in this section while also making use of `Constants` and `HealthMetrics`. Remember to test the functionality of your implementations in `main.cpp`.

Note that for the remainder of this practical “macro”/ “macros” is short for macronutrients. You are not expected to use C++ Macros.

4.1 HealthMetrics

HealthMetrics is a namespace that includes functions for calculating key health indicators. The headers for these functions are given to you in the `HealthMetrics.h` file. Inside the `HealthMetrics.cpp` file, you will have to implement **one of them** as specified.

4.1.1 Functions

The functions below (4.1.1.1 - 4.1.1.3) have been implemented, do not change them.

4.1.1.1 `double bmi(double weightKg, double heightM)`

- This function calculates the BMI, a measure of weight relative to height.

4.1.1.2 `double bmr(int age, double weightKg, double heightM)`

- This function calculates the BMR, the number of kiloJoules the body burns at rest.

4.1.1.3 `double tdee(double bmrValue, int activityLevel)`

- This function calculates Total Daily Energy Expenditure(TDEE) by adjusting BMR for activity level (1 = low, 2 = moderate, 3 = high), giving an estimate of daily energy needs.

4.1.1.4 `int bmiCategory(double bmiValue)`

- This function classifies a BMI value into standard categories such as underweight, normal, overweight or obese.
- The table below shows how the BMI is categorised:

Condition	$value \leq 0$	$0 < value < 18.5$	$18.5 \leq value < 25$	$25 \leq value < 30$	$value \geq 30$
Category	0	1	2	3	4

Table 1: Mapping of BMI value ranges to category

4.2 Constants

Constants is a namespace (in `Meal.h`) that contains string constants (kilojoules, protein, carbohydrates and fat) that are used throughout the practical to avoid issues regarding misspellings. It is not a requirement to use it but it can help.

Usage: `if(macro == PROTEIN)` is the same as `if(macro == "protein")`

4.2.1 Variables

4.2.1.1 `const string KJ`

- KJ represents the text “kJ” which is the symbol for energy in kilojoules.

4.2.1.2 `const string PROTEIN`

- PROTEIN represents the text “protein”.

4.2.1.3 `const string CARBS`

- CARBS represents the text “carbs”.

4.2.1.4 `const string FAT`

- FAT represents the text “fat”.

4.3 Meal

The following functions are given to you in the `Meal.h` file. Inside the `Meal.cpp` file, you will have to implement **two** of them as specified. Note that every `const string meal` parameter is a meal created using `makeMeal` (see below), not just the meal’s name.

4.3.1 Functions

4.3.1.1 `string makeMeal(const string name, double kilojoules, double proteinG, double carbsG, double fatG)`

- **This function has been implemented, do not change it.**
- This function constructs a meal based on the inputted values and stores it as a string in the following format:
meal name | energy in kiloJoules | protein in grams | carbs in grams | fat in grams
- An example of a meal would be:
Chicken Shawarma | 2600kJ | 35g protein | 80g carbs | 25g fat

4.3.1.2 `string getField(const string meal, const string field)`

- **This function has been implemented, do not change it.**
- This function extracts a specific component of a meal string created by `makeMeal`. The `field` parameter can be the meal’s name, energy(kJ), protein, carbs or fat. It returns the value as a string.
- For example, given:
meal = “Chicken Shawarma | 2600kJ | 35g protein | 80g carbs | 25g fat” and
field = PROTEIN, the function returns “35”. *Hint: Constants usage*

4.3.1.3 `double getAmount(const string meal, const string field)`

- **This function has been implemented, do not change it.**
- This function converts the result of `getField` into a `double` value, if possible. It returns the numeric value of the specified field (energy, protein, carbs or fat). If the field is missing or invalid, it returns 0.
- Example: `getAmount(meal, PROTEIN)` would return 35 for the example meal above.

4.3.1.4 `double kjPerGram(const string macro)`

- **This function has been implemented, do not change it.**
- This helper function returns the energy content of one gram of a macronutrient. Protein and carbs provide 17 kJ per gram, while fat provides 37 kJ per gram. An unknown macro is 0 kJ.

4.3.1.5 `void printMealInfo(const string meal)`

- **This function is purely for displaying and has been implemented, do not change it.**
- Prints a meal's details in a readable format, including name, energy, protein, carbs and fat.
- Example output for the meal above:
Meal Name: Chicken Shawarma
Energy (kJ): 2600
Protein (g): 35
Carbs (g): 80
Fat (g): 25

4.3.1.6 `double macroRatio(const string meal, const string macro)`

- This function calculates the proportion of a meal's total energy that comes from a specific macronutrient, given that the meal's total kJ are a natural number.
- It multiplies the macronutrient grams by their kJ per gram and divides by the meal's total energy.
- Example: `macroRatio(meal, PROTEIN)` would return 0.228846 for the meal above. DO NOT return the percentage (i.e. 22.88 %).

4.3.1.7 `bool isHighMacro(const string meal, const string macro)`

- This function determines if a meal is high in a particular macronutrient based on thresholds: protein > 25%, carbs > 50%, fat > 30% of total energy.
Hint: Use doubles when checking (i.e. 0.25 not 25%).
- It returns `true` if the macro ratio strictly exceeds the threshold, otherwise `false`. It also returns `false` if an unknown macro is provided.

4.4 Workout

The following functions are given to you in the `Workout.h` file. Inside the `Workout.cpp` file, you will have to implement **all but one** of them as specified. For the sake of simplicity, we consider one set of an exercise at a time.

4.4.1 Functions

4.4.1.1 `void printSetSummary(int reps, double weightKg)`

- **This function is purely for displaying and has been implemented, do not change it.**
- Displays a summary of a workout set including the number of reps, weight lifted and intensity level.
- Example output:

```
Workout Set: 10 reps @ 50 kg
Intensity Level: 2 (Moderate)
```

4.4.1.2 `bool isSetValid(int reps, double weightKg)`

- This function checks whether a workout set is valid based on the number of repetitions and the weight lifted.
- Valid sets include the following intervals:
 - $[1, 50]$ reps
 - $[0, 300]$ kg of weight
- Extreme combinations (more than 30 reps with over 200 kg) are invalid.
- It returns `true` if the set is valid, otherwise `false`.

4.4.1.3 `char repCategory(int reps)`

- This function categorises a set based on the number of repetitions: ‘S’ for strength training, ‘M’ for muscle building, ‘E’ for endurance.
- The table below shows the rep counts used to determine the category.

4.4.1.4 `int scoreFromCategory(char category)`

- This function converts a rep category into a numeric score.
- The table below shows the category used to determine the score.

Repetition Interval	Category	Score
(0;5]	S	30
(5;12)	M	20
[12; ∞)	E	10
Anything else	N	0

Table 2: Mapping of repetition count to category and corresponding score

4.4.1.5 `int intensityLevel(int reps, double weightKg)`

- This function calculates the intensity level of a set on a scale of 1 (light) to 3 (heavy). It does this by adding the category score to the quotient of `weightKg` and 3.0.
- Intensity levels are categorised by the **condition swapping** of the following:
 - 1 : intensity > 12
 - 2 : intensity > 22
 - 3 : intensity <= 22
- If the set is not valid, the intensity level is 0 and if `weightKg` is 0, the intensity is 1.
- Note that the **intensity** is the calculation done within the function and **intensity level** is what the function returns based on intensity ranges.

4.4.1.6 `int progressionDecision(double prevWeight, double currWeight, int prevReps, int currReps)`

- The first version of this function evaluates the progression of a workout based on both weight lifted and repetitions performed. It compares previous and current weight used and reps.
- Returns 1 if both weight and reps increased or weight increased but reps dropped or remained the same. It returns -1 if both weight and reps decreased or weight decrease but the reps increased or remained the same. It returns 0 otherwise.
- If any inputs are less than 0 or any sets are invalid, return 0.

4.4.1.7 `int progressionDecision(int prevReps, int currReps)`

- The second version of this function evaluates the progression of a workout based on intensity level.
- It returns 1 if intensity level increased and -1 if intensity level decreased or remained the same.
- Reps that are not natural numbers have a progression of 0.

4.5 Gym

The following functions are given to you in the `Gym.h` file. Inside the `Gym.cpp` file, you will have to implement **all but one** of them as specified.

4.5.1 Functions

4.5.1.1 `string goalAdvice(int, double, double, string, string, string, int, char, double, double, int, int)`

- **This function has been implemented, do not change it.**
- This function generates personalised advice by combining health metrics, nutrition analysis and workout progression.
- It is dependent on your implementation of `HealthMetrics`'s `bmiCategory(double bmiValue)` so ensure that it works correctly.

4.5.1.2 `bool isGoalValid(char goal)`

- This function checks whether the provided goal is valid. A goal is considered valid if it is one of the following: 'B' (bulking), 'C' (cutting), or 'T' (mainTenance).
- It returns `true` if the goal is valid, and `false` otherwise.

4.5.1.3 `double kiloJouleTarget(double tdeeValue, char goal)`

- This function calculates the target daily energy intake based on the TDEE and the user's goal.
- For a bulking goal, an additional 500 kJ is added to the TDEE, while for a cutting goal, 500 kJ is subtracted. Maintenance keeps the value unchanged.
- If the TDEE is negative or the goal is not valid, return 0.

4.5.1.4 `double proteinTarget(double weightKg, char goal)`

- This function determines the daily protein intake target based on body weight and goal.
- A multiplier is applied to the weight value as follows:
Bulking: 2.0
Cutting: 1.2
Maintaining: 0.8
- The function returns 0 if weight is negative or goal is not valid.

4.5.1.5 `int nutritionScore(int age, double weightKg, double heightM, const string m1, const string m2, const string m3, int activityLevel, char goal)`

- This function calculates an overall nutrition score using three meals, user health metrics, a fitness goal and any existing functions.
- First it adds up the total energy(kJ) of all the meals and the total protein content.
- Then it determines the kiloJoule target and protein target and calculates a base score based on those as follows:
 - For energy:
 - * Increase the score by 2 if the amount consumed exceeds or is equal to the target.
 - * Increase the score by 1 if the amount consumed exceeds or is or equal to `target * 0.75`.
 - For protein:
 - * Increase the score by 2 if the amount consumed exceeds or is or equal to the target.
 - * Increase the score by 1 if the amount consumed exceeds or is or equal to `target * 0.75`.
 - Additional scoring is applied based on the quality of macronutrient distribution. If bulking, add 1 for each meal that has a high protein ratio. If cutting, subtract 1 for each meal that has a high fat ratio.
- If the final score is below zero, return 0.

5 Implementation Details

- You must implement the functions in the header files exactly as stipulated in this specification. Failure to do so will result in compilation errors on FitchFork.
- You may only use `c++98`.
- You may only utilise the specified libraries. Failure to do so will result in compilation errors on FitchFork.
- You may include using `namespace std` in any of the files.
- You may only use the following libraries:
 - `<string>`
 - `<sstream>`
 - `<iostream>`
- You are supplied with a **trivial** main demonstrating the basic functionality of this assessment.

6 Upload Checklist

The following c++ files should be in a zip archive named uXXXXXXXXX.zip where XXXXXXXXX is your student number:

- HealthMetrics.cpp
- Meal.cpp
- Workout.cpp
- Gym.cpp
- main.cpp
- Any textfiles used by your main.cpp

The files should be in the root directory of your zip file. In other words, when you open your zip file you should immediately see your files. They should not be inside another folder.

7 Submission

You need to submit your source file(s), as a single archive, on the FitchFork website (<https://ff.cs.up.ac.za/>). All methods need to be implemented (or at least stubbed) before submission. Your code should be able to be compiled with the following command:

```
g++ -std=c++98 -Werror -Wall *.cpp -o main
```

1

and run with the following command:

```
./main
```

1

Remember your h files will be overwritten, so ensure you do not alter the provided h files.

You have 5 submissions and your last mark will be your final mark. Upload your archive to the Practical 4 slot on the FitchFork website. There are no late submissions for this practical therefore you should ensure that you submit in a timely manner.

No submissions after the deadline will be accepted!